



EXERCISE · MODULE 2 OF 7

From Inspected-In to Designed-In

A hands-on exercise for Quality-first thinking — redesign a real feature spec so quality is built in, not bolted on.

20 min build

5 min share

5 min debrief

EXERCISE FLOW

Three phases, thirty minutes

01



Redesign

20 min

Small groups rewrite a flawed feature spec, applying quality-first thinking to four dimensions.

02



Share-out

5 min

Two groups present one redesigned dimension each; rest post their work for a gallery walk.

03



Debrief

5 min

Facilitator connects the patterns back to the core principle and previews Module 3.

THE SCENARIO

Hand each group this spec, as written

TICKET-482

Add a “Buy Now” button to the product page

- Add a button labeled “Buy Now” to the product detail page.
- Clicking it takes the user to checkout with the item pre-added to the cart.
- QA will test it before release.
- Ship by Friday.

WHAT'S MISSING

- ✗ No testable definition of “correct”
- ✗ Zero edge cases named
- ✗ Quality = QA's job, after the fact
- ✗ “Done” means merged, not verified

YOUR TASK • 20 MIN • GROUPS OF 3–4

Redesign the spec so quality is built in

Answer all four prompts on the shared doc / whiteboard. Every answer must be specific enough that someone outside your group could act on it.

1 Testable acceptance criteria

Rewrite “clicking it takes the user to checkout” as Given/When/Then criteria a machine could verify.

2 Name the edge cases

List 3 failure modes the spec ignores. For each, decide who (or what) catches it — before code is written.

3 Pick one automated gate

Name one automated check (unit, contract, e2e, or a monitor) that must pass before this ships.

4 Rewrite “Done”

What does “done” mean when quality is designed in — not inspected in by someone else, later?

FACILITATOR RUN SHEET

How to run the room

0:00

Post the ticket (Slide 3) and the four prompts (Slide 4). Form groups of 3–4. Start a visible timer.

0:15

Halfway check-in: if a group has skipped edge cases or automation, nudge them — don't answer for them.

0:20

Time's up. Two groups present one dimension each, 2–2.5 min apiece. Everyone else posts their doc/board.

0:25

Debrief: name the pattern (Slide 11), tie back to the principle, preview Module 3.

Materials: printed/linked ticket, shared doc or whiteboard per group, visible countdown timer.



MODEL SOLUTION

What quality-first actually looks like

The next four slides walk the same “Buy Now” spec through each prompt, step by step — use them to calibrate groups during share-out, not to hand out in advance.

STEP 1 OF 4 · TESTABLE ACCEPTANCE CRITERIA

Turn “works correctly” into something a machine can check

BEFORE

“Clicking it takes the user to checkout with the item pre-added to the cart.”

No stated behavior for stock, price, or session state. “Correct” is whatever QA happens to try.

AFTER

Given a signed-in user on an in-stock product page, when they tap Buy Now, then the cart holds exactly 1 unit of that SKU and checkout loads in <2s.

Given an out-of-stock product, when tapped, then the button is disabled and shows “Out of stock” — no checkout call fires.

Given an expired session, when tapped, then the user is routed to login and returned to checkout after auth.

Why it matters: Given/When/Then criteria become the automated test suite itself — not a checklist someone eyeballs after the fact.

STEP 2 OF 4 · EDGE CASES, NAMED AT DESIGN TIME

Surface failure modes before a line of code exists

BEFORE

The spec names zero edge cases. Whoever writes the code decides, silently, what happens on a double-click or a stock race — and QA has to guess what to try.

AFTER

Double-tap / double submit → idempotency key on add-to-cart. Owner: eng. Test: unit.

Price changes between page load and click → re-validate price server-side before checkout. Owner: eng. Test: contract.

Inventory hits zero mid-click (race) → optimistic lock + friendly retry copy. Owner: eng + product. Test: integration.

Why it matters: Every edge case gets an owner and a test type up front — quality isn't a search QA runs later, it's a list engineering designs against.

STEP 3 OF 4 · AUTOMATED GATES, NOT A MANUAL PASS

Move the check into the pipeline, not a person's inbox

BEFORE

“QA will test it before release.” One manual pass, after the work is “done,” by someone who didn't build it and is easy to skip under deadline pressure.

AFTER

Unit tests for cart logic run on every commit.

Contract test validates the checkout API before merge.

E2E smoke test (buy-now happy path) runs in CI before deploy.

Canary release to 5% of traffic with automatic rollback if error rate > 1%.

Why it matters: Gates that live in the pipeline can't be forgotten, rushed, or skipped — nothing ships that fails them, no reminder required.

STEP 4 OF 4 · A DEFINITION OF DONE THAT INCLUDES QUALITY

Redefine “done” so it can't mean “merged”

BEFORE

“Ship by Friday.” Done = deployed. Nothing says the team will know if it actually works once real users touch it.

AFTER

All automated gates (Steps 1–3) pass.

Dashboard alert configured for a buy-now conversion drop >10%.

Runbook entry exists for on-call if checkout errors spike.

Feature flag in place for an instant rollback.

Why it matters: “Done” now means built, verified, observable, and reversible — quality stays true after launch, not just at merge time.

DEBRIEF · 5 MIN

Three questions to ask at design time, every time

1

What would “correct” mean, testably?

Turns intent into acceptance criteria a machine can run.

2

What breaks this, and who owns catching it?

Names edge cases and assigns them before code exists.

3

How will we know it's working after we ship?

Puts monitoring and rollback inside the definition of done.

The pattern: quality-first isn't extra process — it's the same decisions, made earlier, by the people who understand the risk best.



UP NEXT

Module 3: Automating the gates you just designed

Keep today's redesigned spec — you'll wire its acceptance criteria into an actual CI pipeline next session.